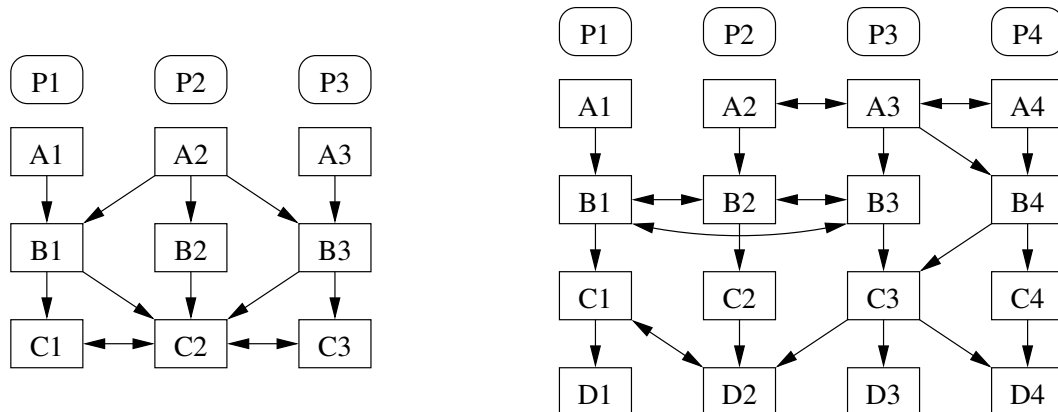


Travaux dirigés n° 6

Les IPC

Exercice 1 (Sémaphores “généralisés”)

Pour chacun des graphes suivants, proposez une solution en utilisant les sémaphores “généralisés”.



Exercice 2 (Logueur)

Nous souhaitons écrire une application de logs à l’aide d’une file de messages. Le *logueur* crée la file et se met en attente de requêtes de la part d’applications, que nous appellerons les *clients*. Un log est caractérisé par un identifiant unique, une date, un code d’application et un message de 256 caractères maximum. Via la file de messages, les clients peuvent ajouter des logs (l’identifiant et la date sont générés par le *logueur*), récupérer les identifiants des logs stockés par le *logueur* ou récupérer un log à partir de son identifiant. Nous utiliserons le modèle de communication client/serveur.

- 1°) Comment les clients peuvent-ils récupérer les réponses à leurs requêtes sans lire celles des autres clients ? Comment empêcher le serveur de les lire ?
- 2°) Pourquoi il n’est pas possible d’envoyer les identifiants de tous les logs en une seule fois ? Proposez une solution.
- 3°) Décrivez les échanges pour les différentes opérations. Vous n’oubliez pas de gérer les erreurs.
- 4°) Comment le *logueur* peut-il recevoir des messages de tailles différentes ? Proposez une solution.
- 5°) Proposez les structures en C nécessaires pour les différents messages.
- 6°) Donnez le code C permettant au client d’envoyer un nouveau log.
- 7°) Donnez le code C permettant au serveur de lire les requêtes (ne traitez que le cas avec l’ajout d’un log).
- 8°) Que se passe-t-il si le nombre de requêtes est trop important ? Quelle solution pouvez-vous proposer ?

Exercice 3 (Les monstres)

Nous souhaitons réaliser un simulateur de monstres. La simulation s’exécute sur une grille de taille variable. Sur chaque case, il ne peut y avoir qu’un seul de ces éléments : un rocher (un obstacle), de la viande, un monstre ou de l’herbe. Chaque monstre se déplace aléatoirement sur l’herbe de case en case puis, s’il rencontre un monstre ou de la viande sur une case adjacente (haut, bas, droite ou gauche), il réalise une action :

- Un autre monstre : dans ce cas, il le tape. Ce monstre perd entre 1 et 10 points de vie.
- De la viande : il en mange un morceau et gagne 10 points de vie. La viande disparaît si le monstre mange le dernier morceau.

La viande est générée régulièrement à une position aléatoire sur la grille avec un nombre aléatoire de morceaux (entre 1 et 10). Les monstres commencent à 100 points de vie et meurent s'ils n'ont plus de points de vie.

Nous souhaitons réaliser plusieurs applications qui communiquent en utilisant les outils IPC :

- *Monstre* : elle permet de créer un monstre sur la grille puis le déplace aléatoirement. Elle s'arrête lorsque le monstre meurt ou quand l'utilisateur la stoppe.
- *Coordinateur* : l'application principale qui met en place les outils IPC et qui affiche la simulation régulièrement. Elle génère également la viande sur la grille.

1°) Modélisation et données.

- a) Quelles données sont nécessaires dans votre application ?
- b) Donnez les structures en C permettant de représenter les différents éléments de l'application.

2°) **Le segment de mémoire partagée.** Nous supposons que le segment de mémoire partagée contient la grille puis les informations sur les monstres (positions et points de vie).

- a) Donnez le contenu du segment de mémoire partagée.
- b) Donnez une structure permettant de mapper le segment de mémoire.
- c) Donnez le code de la fonction `segment_t *creer_segment(key_t cle, grille_t grille, int maxMonstres)` où `cle` est la clé IPC du segment, `grille` est la grille et `maxMonstres` est le nombre maximum de monstres.
- d) Donnez le code de la fonction `segment_t *mapper_segment(key_t cle)` qui permet de mapper un segment déjà créé.

3°) **Synchronisation.** Nous supposons que le coordinateur prend en arguments les clés IPC, la largeur et la hauteur de la grille, ainsi que le nombre maximum de monstres autorisé.

- a) À l'aide d'un sémaphore, expliquez comment limiter le nombre d'exécutions du programme *monstre*.
- b) Expliquez ce que doit faire l'application *monstre* au démarrage.

Pour limiter le nombre de sémaphores, nous découpons la grille par zones de 5 cases par 5.

- c) Comment gérer la concurrence sur les cases de la grille à l'aide des sémaphores ? Et lors des batailles ? Et lorsque la viande est boulotée par plusieurs côtés ?
- d) Donnez le code en C permettant de réaliser les différentes opérations.
- e) Quels problèmes peuvent être rencontrés par le *coordinateur* lors de l'affichage ? Proposez une solution.
- f) Donnez le code en C permettant de créer le tableau de sémaphores et de l'initialiser.
- g) Comment gérer l'arrêt de l'application *monstre* si le monstre meurt, tué par un autre monstre ? Et s'il n'a plus de point de vie suite à un déplacement ?

4°) **Fermeture.** Nous souhaitons maintenant gérer un arrêt propre de l'ensemble des applications. Lorsque l'application *monstre* reçoit un signal SIGINT, elle doit s'arrêter proprement. Lorsque l'application *coordinateur* reçoit un signal SIGINT, elle doit faire arrêter toutes les autres applications, supprimer l'ensemble des outils IPC puis s'arrêter.

- a) Expliquez toutes les problématiques liées à l'arrêt des applications.
- b) Proposez toutes les modifications à apporter pour pouvoir gérer l'arrêt.
- c) Nous supposons que l'utilisateur peut cliquer sur un monstre pour le détruire. Proposez une solution pour gérer ce cas.